Table of Contents

Car Modeling Simulation 1
Motion Generation Function 1
RPM Calculation 3
Cumulated Fuel Calculation 3
Plotting 3

Car Modeling Simulation

Author: Jiale Jiang Editor: Yaoyi Peng, Emily Robinson, Hector Rodruguez

```
% load the saved data for some simple reference
load("CarOBD2data.mat","JettaData");
```

Motion Generation Function

this function takes in initial values, specified motion type, and time; this function returns an object with its field filled internally by the function; acceleration is modeled as simple constant acceleration; rpm modeled second order based on acceleration;

```
function motion = Motion(time,state,a,x0,v0)
    time = time - time(1); % Elapsed time since this segment began
    motion = struct();
    k_rpm = 1; % Estimated model coefficient; units: rpm/ft
    if state == "ACCEL"
        motion.distance = x0 + v0 .* time + ...
            0.5 .* a .* time.^2;
        motion.velocity = v0 + a .* time;
        motion.acceleration = a * ones(size(time));
        motion.rpm = k_rpm .* a .*time.^2;
    end
     if state == "DECEL"
        motion.distance = x0+v0 .* time - 0.5 .* a .* time.^2;
        motion.velocity = v0 -a .* time;
        motion.acceleration = -a .* ones(size(time));
        motion.rpm = -k_rpm .* a .*time.^2;
     end
      if state == "IDLE"
        motion.distance = x0 + zeros(size(time));
        motion.velocity = v0+ zeros(size(time));
        motion.acceleration = zeros(size(time));
        motion.rpm = 0;
      end
      if state == "HOLDSPEED"
        motion.distance = x0 + v0 .* time;
        motion.velocity = v0 .* ones(size(time));
        motion.acceleration = zeros(size(time));
        motion.rpm = 0;
      end
```

```
end

% use initial zero conditions
% set required speed to accelerate to
xi = 0;
vi = 0;
v_mid = 20 * 5280/3600; % 20 mph converted to ft/s
Idle_time = 15;

% since the original test has accel&decel time lower than 30 sec,
% we can use it here;
Accel_time = JettaData{1}.total_accel_time;
Const_speed_time = 15;
Deceleration_time = JettaData{1}.total_decel_time;

% extra safety in case we parsed the wrong data, make sure this hw fits
% requirement by accel/decel in less than 30 sec;
if Accel_time >= 30
    error("accel time too long\n");
end

if Deceleration_time >= 30
    error("decel time too long\n");
end

% total motion time;
total_time = Idle_time+Deceleration_time+Accel_time+Const_speed_time;
% simulating our 10Hz data collection;
frequency = 10;
dt = 1/frequency;
total_samples = total_time/dt;

% assigning time interval for different motion;
total_time_vector = 0:dt:total_time;
time_IDLE = 0:dt:Idle_time;
time_accel = Idle_time:dt:Idle_time+Accel_time;
time_const_speed = Idle_time+Accel_time:dt:Idle_time+...
    Accel_time+Const_speed_time;
time_Decel = Idle_time+Accel_time+Const_speed_time:dt:Idle_time+...
    Accel_time+Const_speed_time+Deceleration_time;

% use const average accel/decel calculation from 20mph requirement
accel = v_mid/Accel_time;
decel = v_mid/Deceleration_time;

% call the motion function and generate all motion point vectors;
idle_motion = Motion(time_IDLE,"IDLE",0,xi,vi);
accel_motion = Motion(time_accel,"ACCEL",accel, ...
    idle_motion.distance(end),idle_motion.velocity(end));
const_speed_motion = Motion(time_const_speed,"HOLDSPEED",0,...
    accel_motion.distance(end),accel_motion.velocity(end));
decel_motion = Motion(time_Decel,"DECEL",decel,...
    const_speed_motion.distance(end),const_speed_motion.velocity(end));
```

```
% prints the total distance traveled;
fprintf("total distance traveled: %.1f ft\n", ...
    decel_motion.distance(end)-xi);
```

total distance traveled: 703.8 ft

RPM Calculation

refer to our car test's idle rpm;

```
const_idle_rpm = JettaData{1}.idle_rpm_ave;

% use idle rpm to form a vector for plotting idle motion rpm;
idle_motion.rpm = const_idle_rpm .* ones(size(time_IDLE));
% const speed rpm should be connected to accel rpm endpoint;
const_speed_motion.rpm = accel_motion.rpm(end) .* ...
    ones(size(time_const_speed));
```

Cumulated Fuel Calculation

use average fuel rate from our car test, ave fuel rate is given by: total fuel used / total distance; Assumed exponential cumulative-fuel model for this short simulation; its coefficient is estimated for this simplified simulation.

```
fuel_rate = JettaData{1}.ave_fuel_rate; % gal/mi

function cumfuel_used = fuelUsedOvetime(fuelrate,time,total_fuel)
    k_fuel = 1; % Estimated model coefficient; units: mi/(gal*s)
    cumfuel_used = total_fuel-total_fuel * exp(k_fuel .* fuelrate .* -time);
end
```

Plotting

figure can help open a seperate window for more and larger plots. follows [left, bot, width,height] so same spacing to left and bot edge(100), 900 by 1000 pixel window for larger plots

```
figure('Units','pixels','Position',[100 100 900 1100]);
% plot distance vs time;
subplot(3,2,1);
plot(time_IDLE,idle_motion.distance,"r","LineWidth",1.5);
hold on;
plot(time_accel,accel_motion.distance,"b","LineWidth",1.5);
plot(time_const_speed,const_speed_motion.distance,"g","LineWidth",1.5);
plot(time_Decel,decel_motion.distance,"m","LineWidth",1.5);
title("distance vs time")
xlabel("Time (s)");
ylabel("Distance (ft)");
legend("idle","accel","const","decel", ...
    "Location","southoutside","NumColumns",2);
hold off;

% plot speed vs time;
```

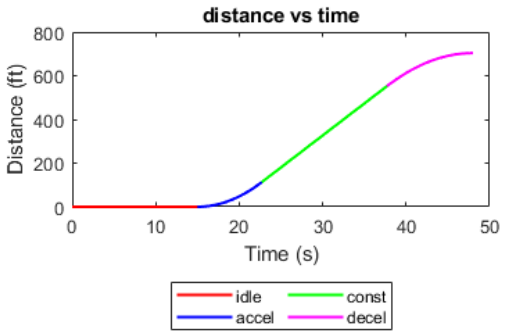
```
subplot(3,2,2);
plot(time_IDLE,idle_motion.velocity,"r","LineWidth",1.5);
hold on;
plot(time_accel,accel_motion.velocity,"b","LineWidth",1.5);
plot(time_const_speed,const_speed_motion.velocity,"g","LineWidth",1.5);
plot(time_Decel,decel_motion.velocity,"m","LineWidth",1.5);
title("velocity vs time");
xlabel("Time (s)");
ylabel("Speed (ft/s)");
legend("idle","accel","const","decel", ...
    "Location","southoutside","NumColumns",2);
hold off;

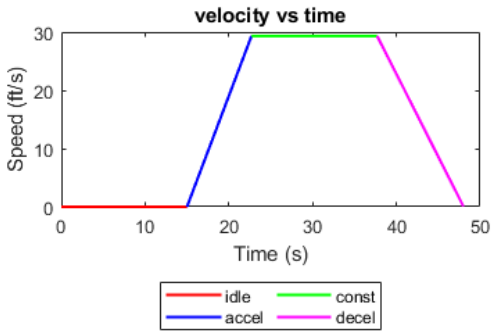
% plot accel vs time;
subplot(3,2,3);
plot(time_IDLE,idle_motion.acceleration,"r","LineWidth",1.5);
hold on;
plot(time_accel,accel_motion.acceleration,"b","LineWidth",1.5);
plot(time_const_speed,const_speed_motion.acceleration,"g","LineWidth",1.5);
plot(time_Decel,decel_motion.acceleration,"m","LineWidth",1.5);
title("acceleration vs time");
xlabel("Time (s)");
ylabel("Acceleration (ft/s^2)");
legend("idle","accel","const","decel", ...
    "Location","southoutside","NumColumns",2);
hold off;

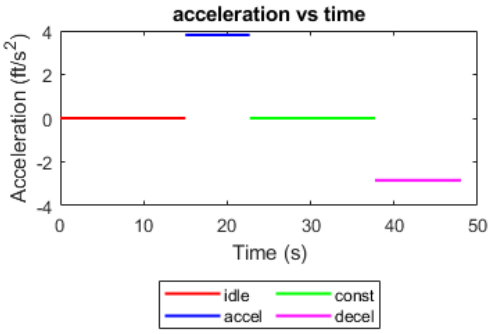
% plotting rpm vs time, continuous plot;
subplot(3,2,4);
plot(time_IDLE,idle_motion.rpm,"r","LineWidth",1.5);
hold on;
plot(time_accel,accel_motion.rpm+const_idle_rpm,"b","LineWidth",1.5);
plot(time_const_speed,const_speed_motion.rpm+ const_idle_rpm,...
    "g","LineWidth",1.5);
plot(time_Decel,decel_motion.rpm + ...
    accel_motion.rpm(end) + const_idle_rpm, ...
    "m","LineWidth",1.5);
title("rpm vs time");
xlabel("Time (s)");
ylabel("Engine speed (rpm)");
legend("idle","accel","const","decel", ...
    "Location","southoutside","NumColumns",2);
hold off;

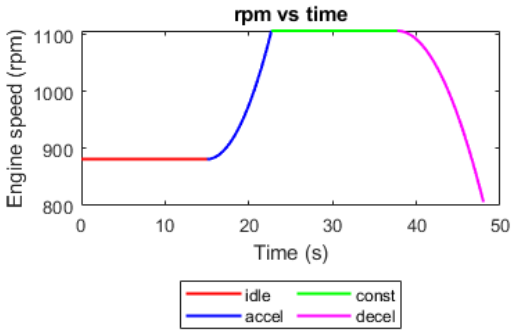
% plot cumulated fuel used vs time;
subplot(3,2,5);
cumulated_fuel_used = fuelUsedOvetime(fuel_rate,total_time_vector,...
    JettaData{1}.TripFuel_gal(end));
plot(total_time_vector,cumulated_fuel_used,"c","LineWidth",1.5);
legend("cumulated fuel used","Location","southoutside");
title("cumulated fuel used vs time");
xlabel("Time (s)");
ylabel("Fuel used (gal)");
fprintf("Total Fuel used: %.6f gal\n",cumulated_fuel_used(end));
```

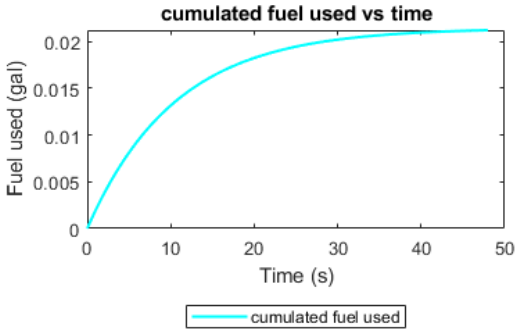
```
Total Fuel used: 0.021209 gal
```











Published with MATLAB® R2024b